

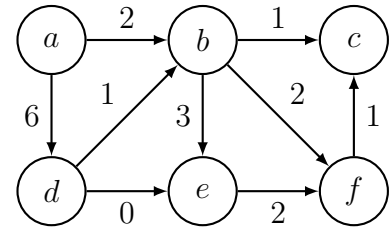
Problem Set 6

Please write your solutions in the $\text{\LaTeX}$ and Python templates provided. Aim for concise solutions; convoluted and obtuse descriptions might receive low marks, even when they are correct.

Problem 6-1. [15 points] Dijkstra Practice

Consider weighted graph $G = (V, E, w)$ below, which is acyclic and has nonnegative edge weights.

- (a) Run both DAG Relaxation and Dijkstra on G from vertex a . Each algorithm tries to relax every edge $(u, v) \in E$, but does so in a different order. For each algorithm, write down a list of all edges in **relaxation order**: the order the algorithm tries to relax them. If there is ambiguity on which vertex or outgoing adjacency to process next, process them in **alphabetical order**.



- (b) List $\delta(a, v)$ for each $v \in V$ (via either algorithm).

Solution: Relaxation order is unique for both algorithms.

DAG Relaxation: $[(a, b), (a, d), (d, b), (d, e), (b, c), (b, e), (b, f), (e, f), (f, c)]$

Dijkstra: $[(a, b), (a, d), (b, c), (b, e), (b, f), (f, c), (e, f), (d, b), (d, e)]$

v	a	b	c	d	e	f
$\delta(a, v)$	0	2	3	6	5	4

Rubric:

- 5 points for each relaxation order (2)
- 5 points for shortest path distances
- -1 point for each mistake (addition, omission, inversion)
- Minimum 0 points for each of three parts

Problem 6-2. [10 points] Short Circuits

Given a weighted directed graph $G = (V, E, w)$ and vertex $s \in V$, with the property that, for every vertex $v \in V$, some minimum-weight path from s to v traverses at most k edges, describe an algorithm to find the shortest-path weight from s to each $v \in V$ in $O(|V| + k|E|)$ time.

Solution: Since this graph may contain cycles and negative weights, at first it seems we do not know how to do better than Bellman-Ford. Fortunately, the problem gives us an additional restriction: that a shortest path to each vertex traverses few edges. Since layer k' of the duplicated

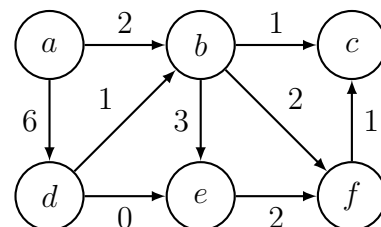
习题集 6

请使用提供的L^AT_EX和Python模板撰写您的解答。力求简洁明了；冗长晦涩的描述即使正确也可能得分较低。

问题6-1. [15分] Dijkstra练习

考虑以下加权图 $G = (V, E, w)$ ，该图无环且边权非负。

- (a) 从顶点 a 开始，对 G 同时运行 DAG 松弛算法和 Dijkstra 算法。每种算法都尝试松弛每条边 $(u, v) \in E$ ，但以不同的顺序进行。对于每种算法，按松弛顺序列出所有边：该顺序算法试图放宽它们。如果存在歧义在处理下一个顶点或出邻接时，按字母顺序处理它们。



- (b) 列出每个 $v \in V$ 的 $\delta(a, v)$ (通过任一算法)。

解决方案：两种算法的松弛顺序是唯一的。

DAG松弛： $[(a, b), (a, d), (d, b), (d, e), (b, c), (b, e), (b, f), (e, f), (f, c)]$ Dij

kstra： $[(a, b), (a, d), (b, c), (b, e), (b, f), (f, c), (e, f), (d, b), (d, e)]$ v

a	b	c	d	e	f	$\delta(a, v)$
0	2	3	6	5	4	

评分标准：

- 每个松弛阶次5分 (2)
- 最短路径距离的5分
- 每个错误（添加、遗漏、颠倒）扣1分
- 三部分每部分最低0分

问题6-2. [10分] 短路

给定一个加权有向图 $G = (V, E, w)$ 和顶点 $s \in V$ ，且满足对于每个顶点 $v \in V$ ，从 s 到 v 的某条最小权重路径最多经过 k 条边，请描述一种算法，在 $O(|V| + k|E|)$ 时间内求出从 s 到每个 $v \in V$ 的最短路径权重。

解法：由于该图可能包含环和负权边，起初我们似乎无法找到比贝尔曼-福特算法更好的方法。幸运的是，问题给出了一个额外的限制条件：到达每个顶点的最短路径经过的边数很少。由于复制层 k' 的

graph G' of Bellman-Ford corresponds to shortest paths to that vertex using at most k' edges, our approach will be to modify Bellman-Ford to only construct the duplicated graph layers incrementally, relax edges into the new layers online to compute k' -edge shortest paths, and then stop when k' -edge shortest path distance do not change from layer to layer.

Specifically, rather than creating the duplicated graph G' all at once and then running DAG Relaxation, we begin with G' only containing the first layer (v_0 for $v \in V$) and δ initialized with first-layer distances $\delta(s_0, v_0) = \infty$ for $v \in V$, except for $\delta(s_0, s_0) = 0$.

Then for i starting at 0, assume for induction that G' contains layers 0 to i and $\delta(s_0, v_j)$ have been computed for all $v \in V$ and $j \in \{0, \dots, i\}$, append layer $i + 1$ by adding vertices v_{i+1} for $v \in V$ and associated edges from layer i . This new layer cannot effect distances in previous layers (since the new layer comes later in the topological order. So, continue DAG Relaxation by relaxing the new edges out of the vertices in layer i , which correctly computes $\delta(s_0, v_{i+1})$ for all $v \in V$.

Since a shortest path from s to each vertex uses at most k edges, the distance $\delta(s, v)$ equals the k' -edge distance $\delta_{k'}(s, v) = \delta(s_0, v_{k'})$ for every $k' \geq k$. Thus, as soon as $\delta(s_0, v_{k'}) = \delta(s_0, v_{k'+1})$ for all $v \in V$, then $k' = k$ and we can terminate by outputting $\delta(s, v) = \delta(s_0, v_k)$ for each $v \in V$.

Since every vertex is reachable from s , $|V| = O(|E|)$, and since we only construct $k + 1$ layers of G' before finding two equal layers, and each layer takes at most $O(|E|)$ time to process, this algorithm runs in $O(k|E|)$ time (which is also $O(|V| + k|E|)$). The $|V|$ was originally provided for this problem to allow the input to include disconnected graph, where an initial single-source reachability algorithm could restrict to the subset of the graph reachable from s ; but this term is no longer relevant for connected input graphs.

Rubric:

- 6 points for description of a correct algorithm that runs in $O(k|E|)$ time
- 2 points for a correct argument of correctness
- 2 points for a correct argument of running time
- Partial credit may be awarded
- Max 4 points for a correct $O(|V||E|)$ -time algorithm

Problem 6-3. [20 points] Dynamite Detonation

Superspy Bames Jond is fleeing the alpine mountain lair of Silvertoe, the evil tycoon. Bames has stolen a pair of skis and a trail map listing the mountain's clearings and slopes (n in total), and she wants to ski from the clearing L by the lair to a clearing S where a snowmobile awaits.

- Each **clearing** $c_i \in C$ has an integer **elevation** e_i above sea level.
- Each **slope** (c_i, c_j, ℓ_{ij}) connects a pair of clearings c_i and c_j with a **monotonic** trail (strictly decreasing or increasing in elevation) with positive integer length ℓ_{ij} . Bames doesn't have time to ski uphill, so she will only traverse slopes so as to decrease her elevation.

Bellman-Ford的图 G' 对应使用最多 k' 条边到达该顶点的最短路径，我们的方法将修改Bellman-Ford，仅增量式地构建复制的图层次，在线松弛进入新层次的边以计算 k' 条边的最短路径，并在 k' 条边的最短路径距离逐层不再变化时停止。

具体来说，我们不是一次性创建重复图 G' 然后运行DAG松弛，而是从仅包含第一层（ v_0 对应 $v \in V$ ）的 G' 开始，并将 δ 初始化为第一层距离 $\delta(s_0, v_0) = \infty$ （针对 $v \in V$ ），但 $\delta(s_0, s_0) = 0$ 的情况除外。

然后对于从0开始的 i ，归纳假设 G' 包含第0层到 i ，并且对于所有 $v \in V$ 以及 $j \in \{0, \dots, i\}$ ， $\delta(s_0, v_j)$ 已被计算，通过添加顶点 v_{i+1} （对于 $v \in V$ ）以及来自第 i 层的关联边，追加第 $i+1$ 层。这一新层不会影响前面各层的距离（因为新层在拓扑顺序中位于更后面）。因此，通过松弛第 i 层顶点的新出边来继续DAG松弛，从而正确计算所有 $v \in V$ 的 $\delta(s_0, v_{i+1})$ 。

由于从 s 到每个顶点的最短路径最多使用 k 条边，因此对于每个 $k' \geq k$ ，距离 $\delta(s, v)$ 等于 k' 边距离 $\delta_{k'}(s, v) = \delta(s_0, v_{k'})$ 。因此，一旦对所有 $v \in V$ 有 $\delta(s_0, v_{k'}) = \delta(s_0, v_{k'+1})$ ，则 $k' = k$ ，我们可以通过为每个 $v \in V$ 输出 $\delta(s, v) = \delta(s_0, v_k)$ 来终止。

由于每个顶点都可以从 s 、 $|V|$ 、 $=$ 、 $O(|E|)$ 到达，并且我们在找到两个相等的层之前只构建了 k 、 $+$ 、1层 G' ，而每层处理最多需要 $O(|E|)$ 时间，因此该算法的时间复杂度为 $O(k|E|)$ （同时也是 $O(|V| \cdot |E|)$ ）。最初为该问题提供的 $|V|$ 允许输入包含不连通的图，此时初始的单源可达性算法可以限制在从 s 可达的子图范围内；但对于连通的输入图，这一术语已不再相关。

评分标准：

- 对在 $O(k|E|)$ 时间内运行的正确算法的描述得6分
- 正确性论证得2分
- 正确运行时间论证得2分
- 部分分数可能会被授予
- 对于正确的 $O(|V||E|)$ 时间算法，最多得4分

问题6-3. [20分] 炸药引爆

超级间谍邦姆斯·琼德正从邪恶大亨银趾的阿尔卑斯山巢穴中逃离。邦姆斯偷了一副滑雪板和一张列出山区空地与雪坡（共 n 处）的路线图，她想要从巢穴附近的空地 L 滑雪到停放雪地摩托的空地 S 。

- 每个空地 $c_i \in C$ 都有一个高于海平面的整数海拔 e_i 。
- 每条雪道 (c_i, c_j, ℓ_{ij}) 连接一对空地 c_i 和 c_j ，形成一条单调的路径（海拔严格递减或递增），路径长度为正整数 ℓ_{ij} 。Bames没有时间滑雪上坡，因此她只会沿着海拔降低的方向穿越雪道。

- On her way up the mountain, Bames laced the mountain with **dynamite** at known locations $C_D \subset C$. Detonating the dynamite will immediately change the elevation of clearings $c_i \in C_D$ by a known amount, from e_i to a lower elevation $e'_i < e_i$. The **detonator** exists at clearing $D \in C$ (where there is no dynamite). If she reaches clearing D , she can choose to detonate the dynamite before continuing on.

Given Bames' map and dynamite data, describe an $O(n)$ -time algorithm to find the minimum distance she must ski to reach the snowmobile (possibly detonating the dynamite along the way).

Solution: We can model the mountain as a DAG of trails that Bames may ski downhill, both before and after possible detonation. Construct a graph G_1 with a vertex for every clearing and a directed edge for every slope pointing to the lower clearing; remove any slopes that do not change height along the way (since Bames will not traverse the slope in either direction). Construct a second graph G_2 in the same way as G_1 , except using the modified heights e'_i that represents the state of the mountain after the dynamite explodes. In both graphs, we weight edges by the length l_{ij} of the corresponding slope. Connect G_1 and G_2 into a single graph G by adding one directed edge from node D_1 in graph G_1 to node D_2 in graph G_2 with weight 0 (taking this edge represents blowing up the dynamite); and add a supernode T that has incoming edges of zero weight from nodes S_1 in graph G_1 and S_2 in graph G_2 (representing reaching S by either not detonating or detonating the dynamite respectively).

We can now run DAG Relaxation from node L_1 in graph G to the supernode T ; the shortest such path is the minimum distance to get from L to S , with or without detonation, while respecting that Bames only decreases her elevation. Graph G has $O(n)$ vertices and edges, so DAG Relaxation runs in $O(n)$ time, as desired.

Rubric:

- 6 points for construction of a graph that can be used to solve the problem
- 3 points for description of a correct algorithm
- 2 points for a correct argument of correctness
- 3 points for a correct argument of running time
- 6 points if correct algorithm is efficient, i.e., $O(n)$ -time
- Partial credit may be awarded

Problem 6-4. [10 points] Conservative Cycles

In Wentonian physics, a force field acts on the motion of a particle by requiring a certain positive or negative amount of **work** to move along any path. A force field is **conservative** if the total sum of work is zero **along every closed loop**. Given a discrete force field represented by n possible particle locations ℓ_i and $O(n)$ possible particle transitions¹, where transition (ℓ_i, ℓ_j, w_{ij}) moves a particle along a **directed** path from location ℓ_i to location ℓ_j requiring (positive or negative) integer work w_{ij} , describe an $O(n^2)$ -time algorithm to determine whether the force field is conservative.

¹Note that a transition from ℓ_i to ℓ_j does not imply the existence of a transition from ℓ_j to ℓ_i .

- 在上山途中，巴姆斯在已知位置 $C_D \subset C$ 安放了炸药。引爆炸药会立即将空地 $c_i \in C_D$ 的海从 e_i 降低至已知的较低海 $e'_i < e_{i_0}$ 。起爆器位于空地 $D \in C$ （此处没有炸药）。若她抵达空地 D ，可在继续前行前选择引爆炸药。

给定 Bames 的地图和炸药数据，描述一个 $O(n)$ 时间的算法，以找到她必须滑雪到达雪地摩托的最短距离（途中可能引爆炸药）。

解决方案：我们可以将山脉建模为一个有向无环图（DAG），其中包含 Bames 可能滑雪下坡的路径，无论是在可能的引爆之前还是之后。构建一个图 G_1 ，每个空地对应一个顶点，每条斜坡对应一条指向较低空地的有向边；移除任何沿途高度不变的斜坡（因为 Bames 不会沿任何方向滑行该斜坡）。以相同方式构建第二个图 G_2 ，但使用修改后的高度 e'_i ，表示炸药爆炸后山脉的状态。在这两个图中，我们根据对应斜坡的长度 l_{ij} 为边赋予权重。通过添加一条从图 G_1 中的节点 D_1 到图 G_2 中的节点 D_2 、权重为 0 的有向边，将 G_1 和 G_2 连接成一个单一图 G （走这条边表示引爆炸药）；并添加一个超级节点 T ，该节点具有来自图 G_1 中的节点 S_1 和图 G_2 中的节点 S_2 的零权重入边，分别表示通过不引爆或引爆炸药到达 S 。

我们现在可以从图 G 中的节点 L_1 对超级节点 T 运行 DAG 松弛；该最短路径是从 L 到 S 的最小距离，无论是否引爆，同时遵守 Bames 仅降低海的条件。图 G 有 $O(n)$ 个顶点和边，因此 DAG 松弛的运行时间为 $O(n)$ ，符合要求。

评分标准：

- 构建一个可用于解决问题的图得 6 分
- 正确算法的描述得 3 分
- 正确性论证得 2 分
- 正确运行时间论证得 3 分
- 如果算法高效，即 $O(n)$ 时间内，得 6 分
- 部分分数可能会被授予

问题 6-4. [10 分] 保守循环

在温顿物理学中，力场通过要求粒子沿任意路径移动时需做一定正功或负功来作用于粒子的运动。若沿每个闭合回路的总功为零，则该力场是保守的。给定一个由 n 个可能粒子位置 ℓ 和 $O(n)$ 个可能粒子转移¹表示的离散力场，其中转移 (ℓ, ℓ', w_{ij}) 使粒子沿有向路径从位置 ℓ 移动到位置 ℓ' ，需做（正或负）整数功 w_{ij} ，请描述一个 $O(n^2)$ 时间算法以判断该力场是否为保守的。

¹Note that a transition from ℓ_i to ℓ_j does not imply the existence of a transition from ℓ_j to ℓ_i .

Solution: Construct a graph G with a vertex for each particle location ℓ_i and a directed edge (ℓ_i, ℓ_j) of weight w_{ij} for each transition (ℓ_i, ℓ_j, w_{ij}) . We can refute that the force field is conservative by finding a cycle in G that has non-zero weight. Add a supernode s connected to every $v \in V$ with a directed zero-weight edge (s, v) (which does not add any new cycles to the graph). Then, run Bellman-Ford from s to determine whether there are any negative-weight cycles; then negate all edges weights and run Bellman-Ford from s again to determine whether there are any negative-weight cycles (positive-weight cycles in the original graph). If Bellman-Ford detects either type of cycle, we can return that the force is not conservative (since we have found a cycle with non-zero weight); otherwise no non-zero weight cycles exist, so we can return that the field is conservative. Since we only add one node and $O(|n|)$ edges to a graph with n vertices and $O(n)$ edges, Bellman-Ford will take $O(n^2)$ time, as desired.

Note that it is possible to solve this problem in $O(n)$ time using algorithms we have not learned in this class, but this is beyond the scope of this class. Namely:

- identify the strongly connected components (SCCs) of G in $O(n)$ time via a modified DFS;
- use DFS within each SCC to assign shortest-path length potentials to each vertex;
- and then check each edge of each SCC against these potentials in $O(n)$ time.

Rubric:

- 3 points for description of a correct algorithm
- 2 points for a correct argument of correctness
- 2 points for a correct argument of running time
- 3 points if correct algorithm is efficient, i.e., $O(n^2)$
- Partial credit may be awarded

Problem 6-5. [20 points] SparkPlug Derby

LightQueen McNing is a race car that wants to drive to California to compete in a road race: the annual SparkPlug Derby. She has a map depicting:

- the n intersections in the country, where each intersection x_i is marked with its positive integer **elevation** e_i and whether it contains a **gas station**; and
- the r roads connecting pairs of them, where each road r_j is marked with the positive integer t_j denoting the **driving time** it will take LightQueen to drive along it in either direction.

Some intersections connect to many roads, but the average number of roads at any intersection is less than 5. LightQueen needs to get from Carburetor Falls at intersection s to the SparkPlug Derby race track at intersection t , subject to the following conditions:

- LightQueen's gas tank has a positive integer **capacity** $g < n$: it can hold up to g units of gas at a time (starting full). Along the way, she can refill her tank (by any integer amount) at any intersection marked with a gas station. It takes exactly t_G time for her to fill up 1 unit of gas.

解法：构建一个图 G ，其中每个顶点对应一个粒子位置 ℓ ，每条有向边 (ℓ, ℓ') 的权重为 w_{ij} ，对应每个转移 (ℓ, ℓ', w_{ij}) 。我们可以通过找到 G 中权重非零的环来反驳力场是保守的。添加一个超级节点 s ，通过权重为零的有向边 (s, v) 连接到每个 $v \in V$ （这不会给图增加任何新环）。然后，从 s 运行贝尔曼-福特算法以判断是否存在负权重环；接着将所有边的权重取反，再从 s 运行贝尔曼-福特算法以判断是否存在负权重环（即原图中的正权重环）。如果贝尔曼-福特算法检测到任何一种环，我们可以返回力场不是保守的（因为找到了权重非零的环）；否则不存在非零权重环，因此可以返回该场是保守的。由于我们只向具有 n 个顶点和 $O(n)$ 条边的图添加了一个节点和 $O(|n|)$ 条边，贝尔曼-福特算法将花费 $O(n^2)$ 时间，符合要求。

请注意，使用本课程未教授的算法可以在 $O(n)$ 时间内解决此问题，但这超出了本课程的范围。即：

- 通过改进的深度优先搜索（DFS）在 $O(n)$ 时间内识别 G 的强连通分量（SCC）；
- 在每个强连通分量内使用深度优先搜索，为每个顶点分配最短路径长度势能；
- 然后以 $O(n)$ 的时间检查每个强连通分量的每条边是否满足这些势能。

评分标准：

- 正确算法的描述得3分
- 正确性论证得2分
- 正确运行时间论证得2分
- 如果算法高效，即 $O(n^2)$ ，得3分
- 部分分数可能会被授予

问题6-5. [20分] SparkPlug Derby

LightQueen McNing是一辆想要前往加利福尼亚参加公路赛的赛车：一年一度的SparkPlug Derby。她有一张地图，描绘了：

- 该国共有 n 个交叉路口，每个路口 x_i 都标有其正整数海拔 e_i 以及是否设有加油站；且
- 连接它们的 r 条道路，其中每条道路 r_j 都标有正整数 t_j ，表示LightQueen沿任一方向行驶所需的时间。

一些交叉口连接着多条道路，但任意交叉口的平均道路数量少于5条。LightQueen需要从位于交叉口 s 的化油器瀑布出发，前往位于交叉口 t 的火花塞德比赛道，需满足以下条件：

- LightQueen的油箱有一个正整数容量 $g < n$ ：它最多可以同时容纳 g 单位的汽油（初始为满油）。沿途，她可以在任何标有加油站的路口加油（任意整数加注量）。每加注1单位汽油恰好需要 t_G 时间。

- LightQueen uses gas only when **driving uphill**. Specifically, if she drives on a road from intersection x_i to x_j at elevations e_i and e_j respectively, LightQueen will use exactly $e_j - e_i$ units of gas to travel along it if $e_j > e_i$, and will use zero units of gas otherwise.

Given LightQueen's map, describe an $O(n^2 \log n)$ -time algorithm to return a fastest route to the race that keeps a **strictly positive** amount of gas in her tank at all times (if such a route exists).

Solution: Create a graph G :

- **Vertices:** for each intersection v , add a vertex v_i for every $i \in \{1, \dots, g\}$, where vertex v_i represents being at intersection v with i units of gasoline in her tank; and
- **Edges:** for each bi-directional road $\{u, v\}$ connecting intersections u and v with driving time $t(u, v)$ (let $e(u)$ and $e(v)$ be the elevations of intersections u and v respectively, and without loss of generality assume $e(u) \leq e(v)$), construct the following directed edges:
 1. (Down-hill Road Edges)
 - add a weight- $t(u, v)$ edge (v_i, u_i) for every $i \in \{1, \dots, g\}$ (since traveling in this direction on the road requires no gas).
 2. (Up-hill Road Edges)
 - if $e(u) == e(v)$, add another weight- $t(u, v)$ edge opposite the previous, specifically: (u_i, v_i) for every $i \in \{1, \dots, g\}$;
 - otherwise $e(v) - e(u) = \ell > 0$, so add a weight- $t(u, v)$ edge $(u_i, v_{i-\ell})$ for every $i \in \{\ell + 1, \dots, g\}$ (since traveling uphill costs gas, and she is not allowed to ever run out of gas).
 3. (Gas Station Edges)
 - for each gas station, e.g., at intersection v , add a weight- t_G edge (v_i, v_{i+1}) for every $i \in \{1, \dots, g - 1\}$ (corresponding to filling up the tank by one unit at the station).

There are gr Down-hill Road Edges, at most gr Up-hill Road Edges, and at most $(g - 1)n$ Gas Station Edges, so graph G has gn vertices and $O(g(r + n))$ edges. This graph has the property that any path from vertex s_g to any vertex t_i for $i \in \{1, \dots, g\}$ will be a valid route for LightQueen to leave Carburetor Falls with a full tank and reach SparkPlug Derby without ever running out of gas along the way; and the path's weight corresponds to the time it would take her to get there. So solve SSSP from s_g to find a minimum weight path to each t_i for $i \in \{1, \dots, g\}$, and return a path to one with minimum weight (or return no such path exists). This graph may contain cycles, but has only positive edge weights, so solve SSSP using Dijkstra.

Since the average degree of each intersection is < 5 and every road connects two intersections, then $2r < 5n$ and $r = O(n)$. Since $g < n$ by the problem statement, G has $O(n^2)$ vertices and $O(n^2)$ edges, so takes $O(n^2)$ time to construct. Then Dijkstra takes $O(n^2 \log(n^2) + n^2) = O(n^2 \log n)$ time, while finding the minimum path to any t_i takes $O(n)$ time, leading to $O(n^2 \log n)$ time in total, as desired.

Rubric:

- LightQueen 仅在爬坡时消耗汽油。具体来说，如果她驾驶在从海拔 e_i 的交叉口 x_i 到海拔 e_j 的交叉口 x_j 的道路上，当满足 $e_j > e_i$ 条件时，LightQueen 将恰好消耗 $e_j - e_i$ 单位的汽油，否则消耗零单位汽油。

给定LightQueen的地图，描述一个 $O(n^2 \log n)$ 时间的算法，返回一条始终让油箱保持严格正油量的最快到达比赛路线（若存在这样的路线）。

解决方案：创建一个图 G ：

- 顶点：对于每个交叉口 v ，为每个 $i \in \{1, \dots, g\}$ 添加一个顶点 v_i ，其中顶点 v_i 表示位于交叉口 v 且油箱中有 i 单位汽油；并且
- 边：对于每条连接交叉口 u 和 v 的双向道路 $\{u, v\}$ ，行驶时间为 $t(u, v)$ （设 $e(u)$ 和 $e(v)$ 分别为交叉口 u 和 v 的海高度，且不失一般性假设 $e(u) \leq e(v)$ ），构建以下有向边：
 - 1.（下坡道路边）
 - 为每个 $i \in \{1, \dots, g\}$ 添加一条权重为 $t(u, v)$ 的边 (v_i, u_i) （因为沿此方向行驶无需耗油）。
 - 2.（上坡路边缘）
 - 如果 $e(u) == e(v)$ ，则在相反方向添加另一条权重为 $t(u, v)$ 的边，具体为：对于每个 $i \in \{1, \dots, g\}$ ，添加 (u_i, v_i) ；– 否则 $e(v) - e(u) = \ell > 0$ ，因此对于每个 $i \in \{\ell + 1, \dots, g\}$ ，添加一条权重为 $t(u, v)$ 的边 $(u_i, v_{i-\ell})$ （因为上坡行驶消耗汽油，且不允许她耗尽汽油）。
 - 3.（加油站边缘）
 - 对于每个加油站，例如在交叉口 v ，为每个 $i \in \{1, \dots, g - 1\}$ （对应将油箱加满一个单位）添加一条权重为 t_G 的边 (v_i, v_{i+1}) 。

共有 gr 条下坡道路边，最多 gr 条上坡道路边，以及最多 $(g - 1)n$ 条加油站边，因此图 G 有 gn 个顶点和 $O(g(r + n))$ 条边。该图具有以下性质：从顶点 s_g 到任意顶点 t_i （其中 $i \in \{1, \dots, g\}$ ）的任何路径，都是光皇后从满油状态离开化油器瀑布并到达火花塞德比且途中永不耗尽燃油的有效路线；且路径的权重对应她到达目的地所需的时间。因此，从 s_g 出发求解单源最短路径问题，以找到到达每个 t_i （其中 $i \in \{1, \dots, g\}$ ）的最小权重路径，并返回其中权重最小的路径（若不存在则返回无此类路径）。该图可能包含环，但边权重均为正，因此使用迪杰斯特拉算法求解单源最短路径问题。

由于每个交叉路口的平均度数为 < 5 ，且每条道路连接两个交叉路口，因此 $2r < 5n$ 且 $r = O(n)$ 。根据问题陈述， $g < n$ 有 $O(n^2)$ 个顶点和 $O(n^2)$ 条边，因此构建需要 $O(n^2)$ 时间。随后 Dijkstra 算法需要 $O(n^2 \log(n^2) + n^2) = O(n^2 \log n)$ 时间，而找到任意 t_i 的最短路径需要 $O(n)$ 时间，总计达到 $O(n^2 \log n)$ 时间，符合要求。

评分标准：

- 6 points for construction of a graph that can be used to solve the problem
- 3 points for description of a correct algorithm
- 2 points for a correct argument of correctness
- 3 points for a correct argument of running time
- 6 points if correct algorithm is efficient, i.e., $O(n^2 \log n)$ -time
- Partial credit may be awarded

- 构建一个可用于解决问题的图得6分
- 正确算法的描述得3分
- 正确性论证得2分
- 正确运行时间论证得3分
- 如果算法高效，即 $O(n^2 \log n)$ 时间，则得6分
- 部分分数可能会被授予

Problem 6-6. [25 points] **Johnson's Algorithm**

In this problem, you will implement Johnson's Algorithm to compute all-pairs shortest-path weights in a general weighted directed graph G , as described in Lecture 14. The input to your algorithm will be: a positive integer n representing the number of vertices of your graph identified by consecutive integers 0 to $n - 1$, and a tuple S of triples, where each triple (u, v, w) corresponds to a directed edge from vertex u to v of weight w . Your output should be a length- n tuple D of length- n tuples where $D[u][v] = \delta(u, v)$ for all $u, v \in \{0, \dots, n - 1\}$, except when the input graph contains a **negative-weight cycle** when you should return `None`.

Please implement the `johnson(n, S)` function in the template code provided. Your code template contains working implementations of Bellman-Ford and Dijkstra (using a binary heap as a priority queue) modified from the recitation notes. Note that you will have to construct your own adjacency list and weight function if you would like to use this code. You can download the code template and some test cases from the website.

Solution:

```

1 INF = 99999          # distance magnitudes will not be larger than this number
2
3 def johnson(n, S):
4     D = [[INF for _ in range(n)] for _ in range(n)]
5     Adj = [[] for _ in range(n)]          # construct graph
6     W = {}
7     for (u, v, w) in S:
8         Adj[u].append(v)
9         W[(u, v)] = w
10    Adj.append([i for i in range(n)])      # run bellman-ford from supernode
11    for i in range(n):
12        W[(n, i)] = 0
13    def wf(u, v):    return W[(u, v)]
14    args = bellman_ford(Adj, wf, n)
15    if args is None:
16        return None
17    h, _ = args
18    def wf(u, v):    return W[(u, v)] + h[u] - h[v]
19    for u in range(n):          # compute SSSP on reweighted graph
20        d, parent = dijkstra(Adj, wf, u)
21        for v in range(n):
22            if d[v] < INF:
23                D[u][v] = d[v] - h[u] + h[v]
24    return tuple(tuple(row) for row in D)

```

问题6-6. [25分] Johnson算法

在本问题中，你将实现约翰逊算法，用于计算一般加权有向图 G 中所有顶点对之间的最短路径权重，如第14讲所述。算法的输入包括：一个正整数 n ，表示图的顶点数，顶点由连续整数 0 到 $n - 1$ 标识；以及一个三元组构成的元组 S ，每个三元组 (u, v, w) 对应一条从顶点 u 到 v 的权重为 w 的有向边。输出应为一个长度为 n 的元组 D ，其中包含长度为 n 的元组，且对于所有 $u, v \in \{0, \dots, n - 1\}$ ，有 $D[u][v] = \delta(u, v)$ ；但如果输入图中包含负权环，则应返回 `None`。

请在提供的模板代码中实现 `johnson(n, S)` 函数。您的代码模板包含了从课堂笔记修改而来的 Bellman-Ford 和 Dijkstra（使用二叉堆作为优先队列）的工作实现。请注意，如果您想使用此代码，您需要自行构建邻接表和权重函数。您可以下载该代码。

来自网站的模板和一些测试用例。

解决方案：

```

1 INF = 99999 # 距离值不会大于此数2
3 def johnson(n, S):
4     D = [[INF for _ in range(n)] for _ in range(n)]
5     Adj = [[] for _ in range(n)] # 构建图
6     W = {}
7     for (u, v, w) in S:
8         Adj[u].append(v)
9         W[(u, v)] = w

10 Adj.append([i for i in range(n)]) # 从超级节点运行贝尔曼-福特算法
11 对于 i 在 range(n) 中:
12     W[(n, i)] = 0
13 def wf(u, v): 返回 W[(u, v)]
14 个参数 = bellman_ford(Adj, wf, n)
15 如果 args 是 None:
16     返回 None
17 小时, _ = 参数
18 def wf(u, v): return W[(u, v)] + h[u] - h[v]
19 对于 u 在 range(n) 中: # 计算 SSSP
20 d, 父节点 = dijkstra(Adj, wf, u)
21 对于 v 在 range(n) 中:
22 如果 d[v] < INF:
23     D[u][v] = d[v] - h[u] + h[v]
24 返回 元组(元组(行) 对于 行 在 D 中) 在 重新加权 图上

```

MIT OpenCourseWare
<https://ocw.mit.edu>

6.006 Introduction to Algorithms
Spring 2020

For information about citing these materials or our Terms of Use, visit: <https://ocw.mit.edu/terms>

MIT开放课程 <https://ocw.mit.edu>

6.006 算法导论 2020年春季

For information about citing these materials or our Terms of Use, visit: <https://ocw.mit.edu/terms>